

TrustLink

Documentación del proyecto — guías, contratos on-chain, arreglos y despliegue

Agosto 2026

TrustLink — arreglos v3

1. Correr en Supabase (SQL Editor)

`sql/13_registro_apellido_dni.sql` (después del 01-12 que ya tenías).

2. Eliminar de tu proyecto

`demo/ordenes.html` — sigue apareciendo en tus ZIPs pero ya no se usa (reemplazado hace un tiempo por `demo/mis-pedidos.html`). Ningún archivo lo referencia; es seguro borrarlo si todavía lo tienes.

3. Arreglado

- **Marketplace en blanco / sin sidebar:** reescrito completo con manejo de errores en cada paso — si algo llega a fallar, ahora se ve un mensaje claro en pantalla en vez de una página muda. Si sigue en blanco después de este cambio, abre la consola del navegador (F12) y comparte el error exacto que aparezca ahí — eso apuntará a la causa real en tu base de datos.
- **Login:** logo centrado en el panel derecho (ya no el ícono persona+), con nota visible de las credenciales `vendedor1@trustlink.com` / `Trustlink2026` para probar como vendedor sin aprobación.
- **Registro:** ahora pide Nombre, Apellido, DNI y Teléfono — así la tarjeta de membresía siempre tiene esos datos desde el inicio.
- **Membresía (real y demo):** tarjeta grande, sin la sección "Mis datos" separada en el real (el nombre/DNI se jalen directo del perfil), y los beneficios de cada nivel están SOLO dentro del flip (antes aparecían duplicados también debajo de la tarjeta). En la demo se mantuvo el formulario de datos, tal como pediste.
- **Exportar CSV/Excel con popup:** nuevo botón único "Descargar" que abre un popup para elegir formato, en Auditoría y en la nueva página de Usuarios.

4. Nuevo

- **Gestión de usuarios estilo QuintaOla** (`usuarios-admin.html`): tabla con filtros (búsqueda, rol, estado), avatares con iniciales, badges de rol coloreados, exportar.
- **Modo oscuro** en real y demo: toggle en el sidebar (bajo "Navegación"), persistente en localStorage, sin parpadeo al cargar la página.
- **Responsive mejorado:** sidebar con scroll en móviles, grids de 2 columnas en pantallas pequeñas, sin scroll horizontal.

Guía: activar los contratos on-chain (hash único por venta)

Con esto, **cada venta de la plataforma genera transacciones reales en Arbitrum Sepolia con su hash único**, verificables en Arbiscan, sin que los usuarios necesiten wallet. Una wallet "operadora" del backend firma todo a través de una Edge Function.

Qué hace cada acción

Acción en la plataforma	Transacción en TrustLinkEscrow	Hash guardado en pedidos
Comprar (crear pedido)	<code>crearOrden(vendedor, codigoHash)</code>	<code>tx_crear_hash</code>
Confirmar entrega con código	<code>confirmarEntrega(ordenId, codigo)</code> → libera y acuña SBT	<code>tx_confirmar_hash</code>
Reportar incidencia de un pedido	<code>reportarDisputa(ordenId)</code>	<code>tx_disputa_hash</code>
Superadmin resuelve la disputa	<code>resolverDisputa(ordenId, liberarAlVendedor)</code>	<code>tx_resolucion_hash</code>

El detalle de cada pedido (Mis compras / Ventas) muestra el bloque "Verificable en blockchain" con los hashes y links a Arbiscan. La disputa de un pedido solo toca SU orden en el contrato: los demás pedidos (aunque sean de la misma compra múltiple) no se ven afectados.

Diseño del MVP (importante entenderlo)

- El monto on-chain es **simbólico** (0.00001 ETH por orden). El dinero real del usuario vive en soles dentro de la plataforma; la blockchain actúa como **notario público** de cada estado de la venta.
- Los vendedores aún no tienen wallet propia: su dirección se **deriva determinísticamente** de su UUID (`keccak256("trustlink:vendedor:"+uuid)`). Siempre es la misma para el mismo vendedor, así el contrato de reputación acumula sus ventas, SBTs y compradores distintos de forma consistente y auditable públicamente.
- Si los contratos no están configurados, TODO sigue funcionando igual: el anclaje simplemente se omite (fire-and-forget).

Pasos para activarlo

1. Desplegar los contratos (una sola vez)

Con Remix (remix.ethereum.org) o Hardhat, en **Arbitrum Sepolia** (chainId 421614), usando una wallet con ETH de faucet (<https://faucet.quicknode.com/arbitrum/sepolia>):

1. Despliega `contracts/TrustLinkReputation.sol`.
2. Despliega `contracts/TrustLinkEscrow.sol` pasándole la dirección del de reputación al constructor.
3. En el de reputación, llama `setEscrowContract(direccionDelEscrow)`.

Usa la MISMA wallet para desplegar y como operadora: así es `owner` del Escrow y puede resolver disputas.

2. Correr el SQL

En el SQL Editor de Supabase: `sql/14_pedidos_onchain.sql` (agrega las columnas de hash a la tabla `pedidos`).

3. Desplegar la Edge Function y sus secrets

```
supabase functions deploy escrow-onchain
supabase secrets set OPERATOR_PRIVATE_KEY=0x... # clave privada de la wallet operadora
supabase secrets set ESCROW_ADDRESS=0x...      # dirección del Escrow desplegado
```

La clave privada **solo** vive como Secret en Supabase — nunca en el código del frontend.

4. Vincular el frontend de solo-lectura

En `js/web3-contratos.js` , pega las dos direcciones en `TL_WEB3` (`escrowAddress` y `reputationAddress`). La página **Contratos** de la sidebar empezará a leer datos reales: estado de órdenes, ventas exitosas, SBTs y score de cualquier vendedor.

5. Probar

1. Compra un producto → en unos segundos, abre el pedido en Mis compras: verás "Verificable en blockchain" con el hash de `crearOrden` .
2. Confirma la entrega con el código → aparece el hash de la liberación (y el SBT acuñado al vendedor).
3. En la página Contratos, consulta la orden por su ID o la reputación del vendedor (su dirección derivada) — los datos salen del contrato.

Guía: activar el chatbot con IA (Gemini)

El chatbot funciona SIEMPRE (con respuestas por palabra clave) aunque no configures nada de esto. Esta guía es solo para activar las respuestas con IA real.

Por qué "puse las API keys pero no funciona"

Las keys de Gemini **nunca van en el código del frontend** (HTML/JS que corre en el navegador) — eso las expondría a cualquiera que abra las herramientas de desarrollador. Van como **Secret de la Edge Function**, dentro de Supabase. Si las pegaste en algún archivo `.js` o `.html` del proyecto, por eso no funciona: la Edge Function nunca las ve.

Pasos

1. Desplegar la Edge Function

La función ya está escrita en `supabase/functions/chatbot-ia/index.ts`. Con Supabase CLI instalado y logueado:

```
supabase link --project-ref TU_PROJECT_REF
supabase functions deploy chatbot-ia
```

`TU_PROJECT_REF` es el ID de tu proyecto (lo ves en la URL del dashboard de Supabase: `supabase.com/dashboard/project/ESTE_ID`).

2. Configurar la(s) API key(s) de Gemini como Secret

Consigue una key gratis en <https://aistudio.google.com/apikey>, luego:

```
supabase secrets set GEMINI_API_KEYS=tu_key_aqui
```

Puedes poner varias separadas por coma para tener respaldo si una se queda sin cuota gratuita:

```
supabase secrets set GEMINI_API_KEYS=key1,key2,key3
```

O desde el dashboard: Project Settings → Edge Functions → Secrets → Add secret → nombre `GEMINI_API_KEYS`, valor tu key (o keys separadas por coma).

3. Verificar

Abre cualquier página del sitio con sesión iniciada, abre el chatbot (ícono de Trusti abajo a la derecha) y pregunta algo como "¿cómo funciona el escrow?". Si responde con una redacción distinta cada vez (no siempre el mismo texto fijo), la IA está activa.

Si sigue respondiendo con el set fijo de respuestas por palabra clave, revisa los logs de la función:

```
supabase functions logs chatbot-ia
```

Errores comunes ahí: - Gemini respondió 400 → key inválida o mal copiada (espacios extra). - Gemini respondió 429 → se acabó la cuota gratuita de esa key; agrega otra al pool separada por coma. - Sin logs / función no responde → no se desplegó (repite el paso 1).

Opcional: respaldo con Claude

Si además configuras `ANTHROPIC_API_KEY` como Secret, la función lo usa como segundo respaldo cuando TODAS las keys de Gemini fallan. No es obligatorio — sin este secret, si Gemini falla, el chatbot cae directo al set de respuestas por palabra clave (que siempre funciona).

TrustLink — Arreglos aplicados

Copia estos archivos sobre tu proyecto respetando las rutas (reemplaza los existentes; los `img/membresias/back-*.jpg` son nuevos).

1. Marketplace en blanco (rol comprador) — RESUELTO

`marketplace.html` redeclaraba `let tlPerfilActual`, que ya existe en `js/app-real.js`. Eso lanza un `SyntaxError` que mata todo el script de la página antes de ejecutarse (sin sidebar, sin productos, sin mensaje de error). Se eliminó la redeclaración.

2. Layout estilo QuintaOla (todos los roles)

- Header superior fijo con: logo TrustLink, toggle de modo oscuro, campana de notificaciones y **nombre completo** del usuario con avatar (`js/app-real.js` + `css/layout.css`).
- Sidebar izquierda **fija** (ya no se recorta al hacer scroll ni deja franja blanca).
- Tipografía **Poppins** (como QuintaOla) en todo el sitio.

3. Modo oscuro optimizado

- Variables nuevas en `css/variables.css` con contraste alto.
- Superficies siempre-oscuras (sidebar, header del chatbot, toasts) usan texto blanco fijo: ya se lee el nombre del perfil, "Trusti Chatbot", etc.

4. Logo en el header

El logo de TrustLink ahora vive en la esquina superior izquierda del header fijo, como QuintaOla.

5. Membresías (`membresia.html`)

- Proporciones corregidas (imágenes con `object-fit: cover`, radios y tipografías equilibradas).
- **Nivel garantizado por total comprado**: al llegar a S/300 la tarjeta superior cambia a Silver (S/800 Gold, S/2000 Black) aunque la BD aún no haya recalculado.
- Barra de progreso corregida (antes quedaba en 0% en Silver/Gold).
- La tarjeta solo muestra **Rosell Pretel** y debajo su **DNI**.
- Reversos de las tarjetas con los **degradados** que enviaste (`img/membresias/back-*.jpg`), texto oscuro donde el fondo es claro.

6. Perfil

- Se muestra nombre + apellido en header y sidebar.
- El modal "Mi perfil" permite editar **nombre y apellido** (y teléfono).
- La X de cerrar es ahora un botón circular profesional (usado en todos los modales).

7. Chatbot Trusti

- Nombre "Trusti Chatbot" visible en modo oscuro.
- Alpaca con **anillo degradado giratorio** + anillo punteado externo.

8. Login

- Tarjeta "Cuenta de prueba · Vendedor" con credenciales y botón **Usar** que rellena el formulario.
- Botones con degradado, inputs redondeados, Poppins.

9. Panel admin estilo QuintaOla

- KPIs con íconos de colores, tarjetas de gestión con descripción y badges de pendientes, distribución de roles y actividad reciente (desde auditoría).

10. Sidebar fija + paginación

- Sidebar fija en todos los roles (punto 2).
- `auditoria.html` con **paginación** de 15 filas (helper `tlPaginar` en `js/app-real.js`, reutilizable).

Demo ([demo/](#))

Mismo rediseño con datos hardcodeados y acceso sin contraseña (elige rol al entrar). CSS compartido copiado, header/sidebar nuevos en `demo/js/app.js`, membresía con degradados y datos precargados (Rosell Pretel + DNI), admin enriquecido.

TrustLink - Guia de despliegue

Vercel + Supabase + PostgreSQL

Agosto 2026

1. Qué vas a desplegar

Este paquete tiene **dos partes**:

- `/demo` -- la demo estática del hackathon (sin backend, sin registro). No necesita nada de lo que sigue en esta guía; súbela a Vercel tal como está y funciona sola.
- **La raíz del proyecto** -- la app **funcional real**: cualquier persona se registra, empieza con S/230 de crédito, compra, vende, y todo pasa por una base de datos de verdad (Supabase/PostgreSQL). Esta guía es para desplegar **esa parte**.

El flujo completo que vas a montar:

1. Creas un proyecto en **Supabase** (base de datos + autenticación + almacenamiento de archivos).
2. Corres los archivos SQL que te dejamos, en orden, para crear todas las tablas y reglas.
3. Configuras la función de IA del chatbot (opcional, pero recomendado).
4. Subes el código a **Vercel** con las credenciales de Supabase.
5. Pruebas el flujo completo: registro, compra, entrega, disputa, crédito.

No necesitas saber programar para seguir esta guía -- es copiar, pegar, y hacer clic en los botones indicados.

2. Crear el proyecto en Supabase

1. Ve a supabase.com y crea una cuenta gratuita (puedes usar tu cuenta de GitHub o Google).
2. Haz clic en **"New Project"**.
3. Completa:
 - **Name**: `trustlink` (o el nombre que prefieras)
 - **Database Password**: genera una contraseña fuerte y **guárdala en un lugar seguro** -- la necesitarás si accedes directo a la base de datos alguna vez.
 - **Region**: elige la más cercana a Perú (por ejemplo `South America (São Paulo)` si está disponible, o la más cercana disponible).
4. Haz clic en **"Create new project"**. Espera 1-2 minutos mientras Supabase aprovisiona tu base de datos.

Cuando termine, verás el panel de tu proyecto.

3. Correr el SQL (crear toda la base de datos)

En el menú lateral de Supabase, ve a **SQL Editor** (ícono de consola). Vas a correr **5 archivos, en este orden exacto**, cada uno en una consulta nueva:

1. `sql/01_schema.sql` -- crea todas las tablas
2. `sql/02_funciones.sql` -- crea la lógica de negocio (pedidos, código de confirmación, créditos, etc.)
3. `sql/03_rls.sql` -- activa la seguridad por fila (cada usuario solo ve lo suyo)
4. `sql/05_storage.sql` -- crea el espacio para guardar fotos de incidencias
5. `sql/04_seed_productos.sql` -- **este lo corres después**, más abajo se explica cuándo

Para cada archivo (excepto el último):

1. Abre el archivo `.sql` con cualquier editor de texto (Bloc de notas, TextEdit, VS Code) y copia **todo** el contenido.
2. En Supabase, haz clic en **"New query"**.
3. Pega el contenido completo.
4. Haz clic en **"Run"** (o `Cmd/Ctrl + Enter`).
5. Debe decir **"Success. No rows returned"** (o similar). Si aparece un error, revisa que hayas copiado el archivo completo y que los anteriores ya se hayan corrido bien.

Repite para los archivos 1, 2, 3 y 4 (el de storage), en ese orden.

Verificar que todo se creó bien

Ve a **Table Editor** en el menú lateral. Deberías ver estas tablas: `perfiles`, `bloqueos_compra`, `solicitudes_vendedor`, `productos`, `carrito_items`, `pedidos`, `pedido_eventos`, `incidencias`, `incidencia_imagenes`, `solicitudes_credito`, `notificaciones`, `auditoria`.

4. Crear tu primera cuenta y promoverte a vendedor (para los productos de ejemplo)

Los productos de ejemplo (`04_seed_productos.sql`) necesitan que **ya exista al menos un vendedor** -- así que primero:

1. Vas a desplegar el sitio en Vercel (sección 7) o puedes probarlo localmente abriendo `login.html` en tu navegador apuntando ya a tu Supabase (necesitas completar la sección 5 primero).
2. Regístrate normalmente en tu propio sitio con tu correo.
3. En Supabase, ve a **Table Editor -> perfiles**. Busca tu usuario (por tu nombre) y edita la columna `rol`, cambiándola de `comprador` a `vendedor`.
4. Ahora vuelve al **SQL Editor**, abre `sql/04_seed_productos.sql`, cópialo completo, pégalo y dale **Run**. Esto crea 8 productos de ejemplo asociados a tu cuenta (ahora vendedora).

Si prefieres no hacerlo tú mismo, cualquier otro usuario que se registre y sea promovido a vendedor por el superadmin (ver "Crear la cuenta de superadmin" más abajo, en esta misma sección) también puede recibir estos productos -- simplemente vuelve a correr el script cuando quieras.

Crear la cuenta de superadmin

Por defecto, todo usuario nuevo se registra como `comprador`. Para tener un superadmin:

1. Regístrate con otra cuenta (o usa la misma).
2. En **Table Editor -> perfiles**, busca ese usuario y cambia `rol` a `superadmin`.
3. Cierra sesión y vuelve a entrar -- ahora verás el panel de administración.

5. Configurar las credenciales de Supabase en el código

1. En Supabase, ve a **Project Settings -> API** (ícono de engranaje, luego "API" en el menú).
2. Copia estos dos valores:
 - **Project URL** (algo como `https://abcdefgh.supabase.co`)
 - **anon public** key (una cadena larga que empieza con `eyJ...`)
3. Abre el archivo `js/supabase-client.js` de este proyecto con un editor de texto.
4. Reemplaza estas dos líneas al inicio del archivo:

```
const SUPABASE_URL = 'PEGA_AQUI_TU_SUPABASE_URL';
const SUPABASE_ANON_KEY = 'PEGA_AQUI_TU_SUPABASE_ANON_KEY';
```

con tus valores reales, por ejemplo:

```
const SUPABASE_URL = 'https://abcdefgh.supabase.co';
const SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
```

5. Guarda el archivo.

Importante sobre seguridad: la "anon key" es pública por diseño -- no es un secreto que deba ocultarse (de hecho, cualquier página web que use Supabase la expone en el navegador). La seguridad real está en las políticas RLS que ya corriste en el paso 3, que garantizan que cada usuario solo pueda ver y modificar lo que le corresponde, sin importar quién tenga esta key.

6. (Opcional pero recomendado) Activar la IA del chatbot

El chatbot funciona **sin este paso** -- usa un motor de respuestas predefinidas que cubre las preguntas más comunes. Pero si quieres que responda con IA real (Claude) a **cualquier pregunta libre** sobre la plataforma, sigue esto:

6.1 Instalar la CLI de Supabase (una sola vez)

Necesitas Node.js instalado en tu computadora. Abre la Terminal (Mac) o PowerShell (Windows) y corre:

```
npm install -g supabase
```

6.2 Conectar tu proyecto

```
supabase login
```

Esto abre tu navegador para autenticarte. Luego, dentro de la carpeta de este proyecto:

```
supabase link --project-ref TU_PROJECT_REF
```

El `TU_PROJECT_REF` lo encuentras en la URL de tu proyecto Supabase (la parte antes de `.supabase.co`), o en **Project Settings -> General**.

6.3 Desplegar la función del chatbot

```
supabase functions deploy chatbot-ia
```

6.4 Conseguir una API key de Anthropic

1. Ve a console.anthropic.com y crea una cuenta (o inicia sesión).
2. Ve a **API Keys** y crea una nueva key.
3. Cópiala (empieza con `sk-ant-...`).

6.5 Configurar la key en Supabase

```
supabase secrets set ANTHROPIC_API_KEY=sk-ant-tu-key-aqui
```

O desde el panel web: **Project Settings -> Edge Functions -> Secrets -> Add new secret**, con nombre `ANTHROPIC_API_KEY` y el valor de tu key.

Listo -- el chatbot ahora responde con IA real. Si en algún momento la función falla o se te acaba el crédito de la API, el chatbot **sigue funcionando** con el motor de respuestas predefinidas, sin que el usuario note el cambio.

7. Desplegar en Vercel

7.1 Si nunca has usado Vercel desde la terminal

```
npm install -g vercel
vercel login
```

7.2 Desplegar

Desde la carpeta raíz de este proyecto (donde está `index.html`, `login.html`, etc. -- **no** dentro de `/demo`):

```
cd ruta/a/tu/proyecto
vercel --prod
```

Te va a preguntar: - **"Set up and deploy?"** -> Enter (sí) - **"Which scope?"** -> elige tu cuenta - **"Link to existing project?"** -> No (si es la primera vez) - **"What's your project's name?"** -> escribe el nombre que quieras, ej. `trustlink-app` - **"In which directory is your code located?"** -> `./` (Enter)

Vercel detecta que es un sitio estático (HTML/CSS/JS) y lo despliega automáticamente. Al terminar, te da una URL como `https://trustlink-app.vercel.app` -- esa es tu app funcional real, ya en línea.

7.3 Actualizaciones futuras

Cada vez que cambies algo en el código, solo corre de nuevo:

```
vercel --prod
```

Y se actualiza la misma URL.

8. Probar el flujo completo

Con todo desplegado, aquí tienes un recorrido para probar que todo funciona de punta a punta:

Como comprador (regístrate con una cuenta nueva)

1. Entra a tu URL de Vercel -> **Crear cuenta**.
2. Verás que tienes **\$/230.00** de crédito en el sidebar.
3. Ve a **Marketplace**, elige un producto, **Comprar con escrow**.
4. Ve a **Mis compras** -- verás el pedido en estado "Pago retenido en garantía".
5. Revisa la **campana de notificaciones** (arriba a la derecha) -- ahí está tu código de 6 dígitos guardado para cuando lo necesites.

Como vendedor (usa la cuenta que promoviste en la sección 4)

1. Entra con esa cuenta, ve a **Ventas** (o **Mis pedidos**).
2. Abre el pedido que el comprador acaba de hacer.
3. Márcalo **"Marcar como enviado"** -> **"Marcar en camino"** -> **"Marcar como llegado"**.
4. Al marcarlo como llegado, se dispara automáticamente una notificación al comprador con el código y arranca el conteo de 15 minutos.

De vuelta como comprador

1. Ve a la notificación (o a **Mis compras** -> abre el pedido).
2. Verás el temporizador corriendo. Ingresas el código de 6 dígitos que te llegó por notificación.
3. Si aciertas, el pedido pasa a "Confirmado" y el pago se libera al vendedor.
4. **Para probar el bloqueo de 12 horas:** crea otra compra, marca "llegado" como vendedor, y simplemente espera a que se acabe el timer sin ingresar el código (o edita manualmente `codigo_vence_en` en Supabase a una fecha pasada para probarlo más rápido). Al intentar confirmar tarde, tu cuenta queda bloqueada.

Reportar una incidencia

1. Desde cualquier rol, ve a **Reportar incidencia**.
2. Elige un pedido, una categoría, sube una foto, y envía.
3. Entra con la cuenta de **superadmin** -> **Incidencias** -> ábrela, revisa la foto, y resuélvela a favor de una de las partes.

Crédito (como vendedor)

1. Ve a **Crédito progresivo** -> solicita un monto (ej. \$/100).
2. Como superadmin, ve a **Créditos pendientes** -> apruébalo.
3. Como vendedor, vuelve a **Crédito progresivo** -> **Pagar ahora**. Verás que se paga el monto **con el 8% de interés incluido**.

Auditoría (superadmin)

1. Ve a **Auditoría** -- verás el registro de todo lo que acabas de hacer (pedidos, confirmaciones, incidencias resueltas, créditos).
2. Prueba los filtros de fecha y acción, y el botón **Descargar CSV**.

9. Preguntas frecuentes

¿Necesito pagar algo? Tanto Supabase como Vercel tienen planes gratuitos generosos, suficientes para una demo de hackathon o un proyecto en etapa temprana. Si tu proyecto crece mucho (miles de usuarios activos), eventualmente necesitarás un plan de pago en alguno de los dos.

¿Qué pasa si pierdo la "anon key" o la contraseña de la base de datos? La anon key la puedes volver a copiar cuando quieras

desde **Project Settings** -> **API**. La contraseña de base de datos, si la pierdes, puedes resetearla desde **Project Settings** -> **Database**.

¿Cómo agrego más productos de ejemplo o los borro? Desde **Table Editor** -> **productos** en Supabase puedes editar, agregar o eliminar filas directamente, sin necesidad de SQL.

¿La demo estática (/demo) usa esta base de datos? No. La demo es 100% independiente y simulada -- no toca Supabase para nada. Es la versión "solo mostrar" pensada para el jurado; la app real (raíz del proyecto) es la que usa todo lo de esta guía.

¿Puedo cambiar el interés del 8% de los créditos? Sí -- en `sql/02_funciones.sql`, dentro de la función `solicitar_credito`, busca la línea `round(p_monto * 1.08, 2)` y cambia `1.08` por el multiplicador que quieras (ej. `1.05` para 5%). Tendrás que volver a correr esa función actualizada en el SQL Editor de Supabase.

¿Puedo cambiar el tiempo de 15 minutos o el bloqueo de 12 horas? Sí -- en la misma función, busca `interval '15 minutes'` (en `avanzar_pedido`) e `interval '12 hours'` (en `confirmar_entrega`), y ajusta los valores.